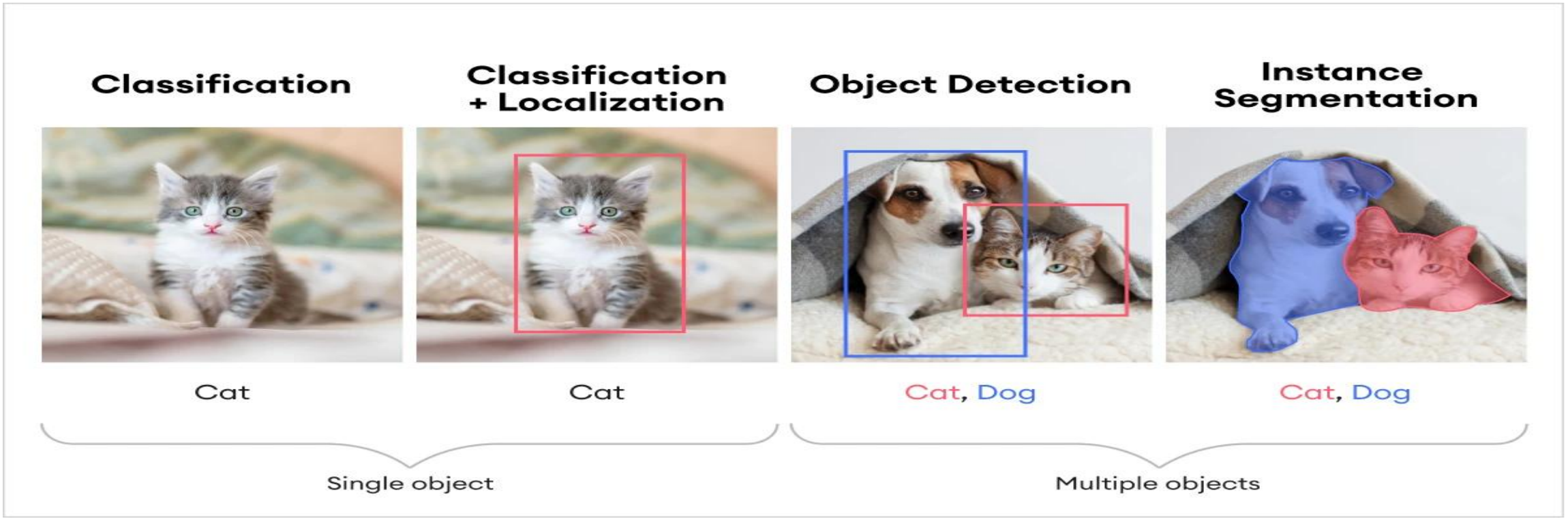
What Are We Trying to Predict?

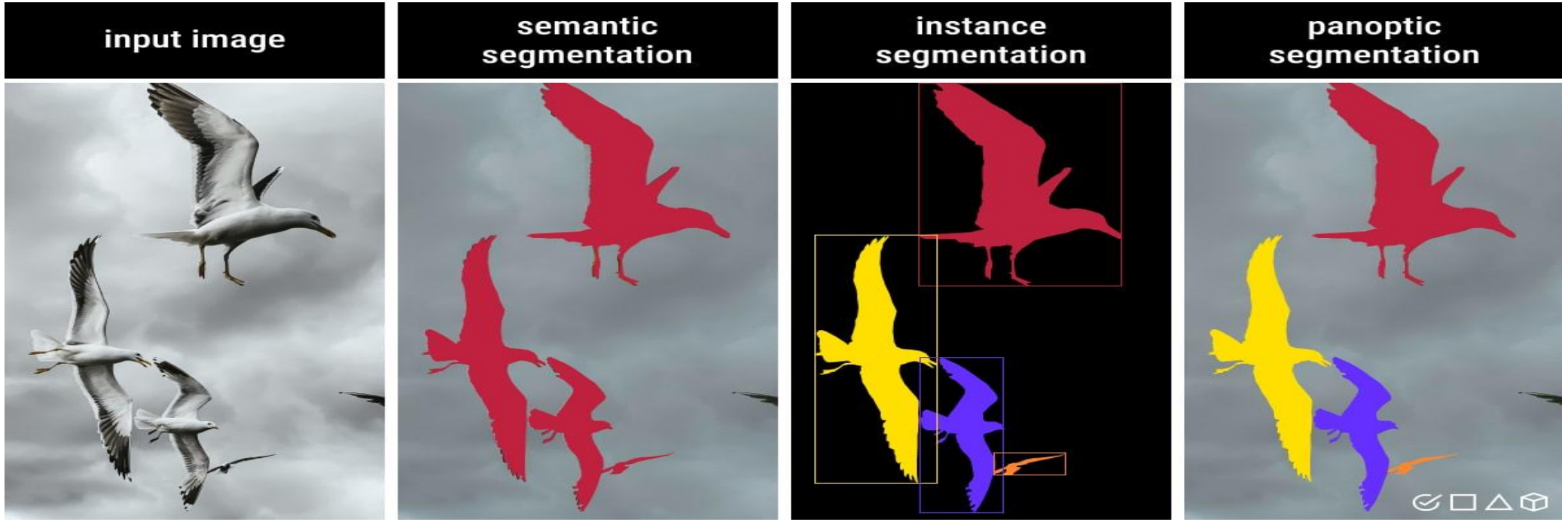


→ ONE class → CLASS + ONE BOX → CLASSES + MANY BOXES → CLASS PER PIXEL

Spatial understanding increases: CLASS → BOX → MANY BOXES → PIXELS

Segmentation Has More Than One Meaning

Once we predict pixels, we can ask whether pixels only need a class—or also an object identity.



Every pixel for similar object gets a same CLASS.

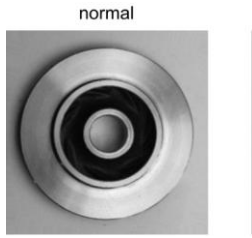
Every object gets its own MASK.

Complete scene understanding: stuff + things.

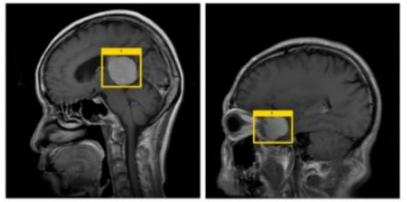
Where Are These Tasks Used?

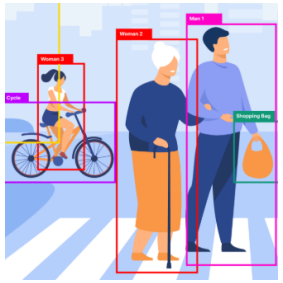
The task is chosen by the question the application needs answered.

Classification	**Defect / no defect**
Localization	**Find tumor / defect**
Detection	**Cars / pedestrians**
Semantic segmentation	**Cars / road / sky / building**
Instance segmentation	**Individual cars**
Panoptic segmentation	**Every region + instance**

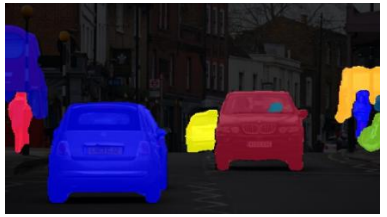














Application decides the task: WHAT do I actually need the model to output?

Why Not Use Segmentation for Everything?

More precise output is not automatically a better engineering choice.

CLASSIFICATION	DETECTION	SEGMENTATION
Small output Low compute Usually cheap annotation	More spatial detail More compute Box annotations	Pixel-level output Higher compute Expensive annotation

ENGINEERING TRADE-OFF

ACCURACY ↔ COMPUTATION ↔ LATENCY ↔ ANNOTATION COST

Choose the simplest way to solve the application.

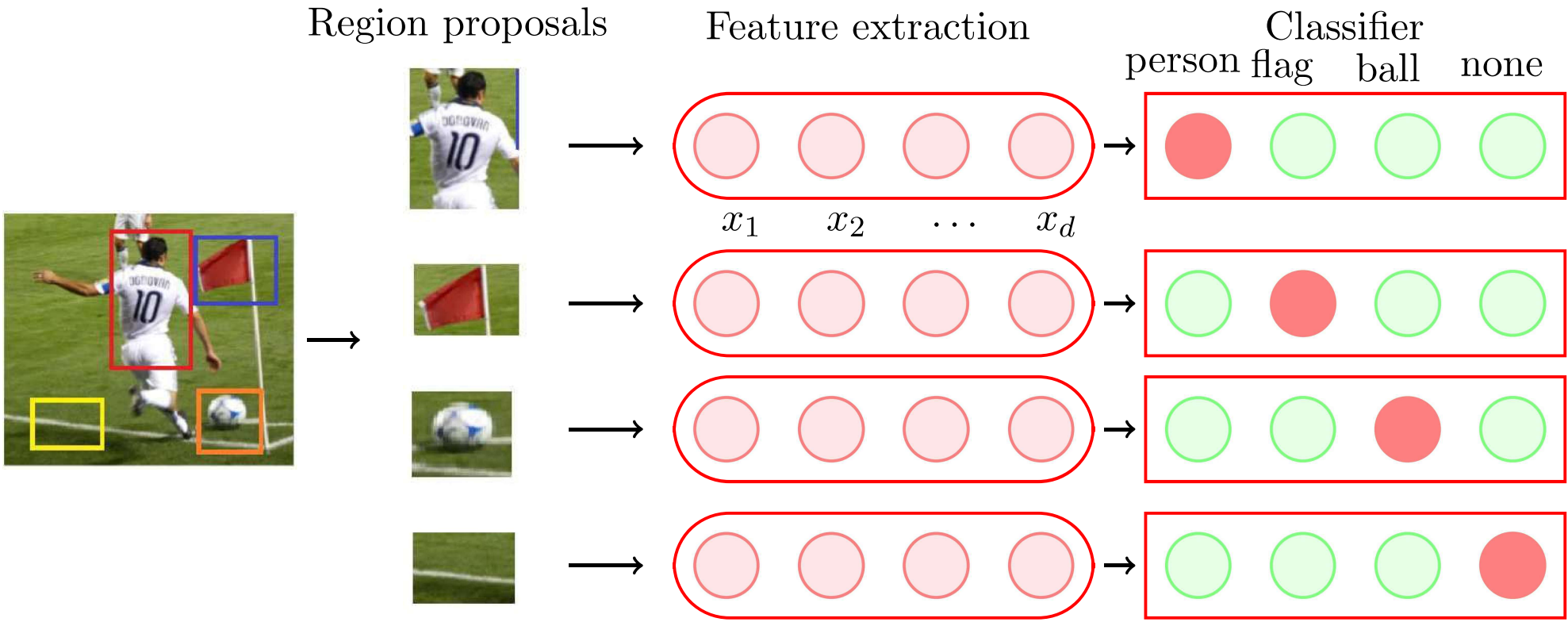
Module 6.1 : Introduction to object detection

- So far we have looked at Image Classification
- We will now move on to another Image Processing Task - *Object Detection*

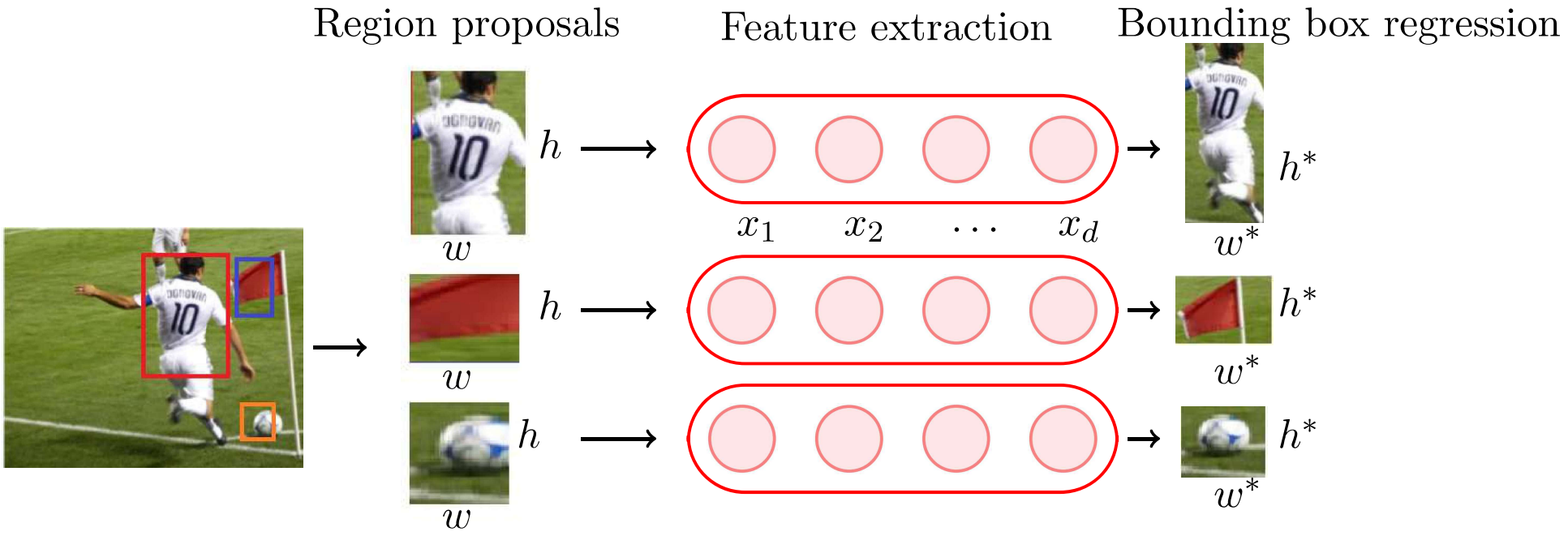




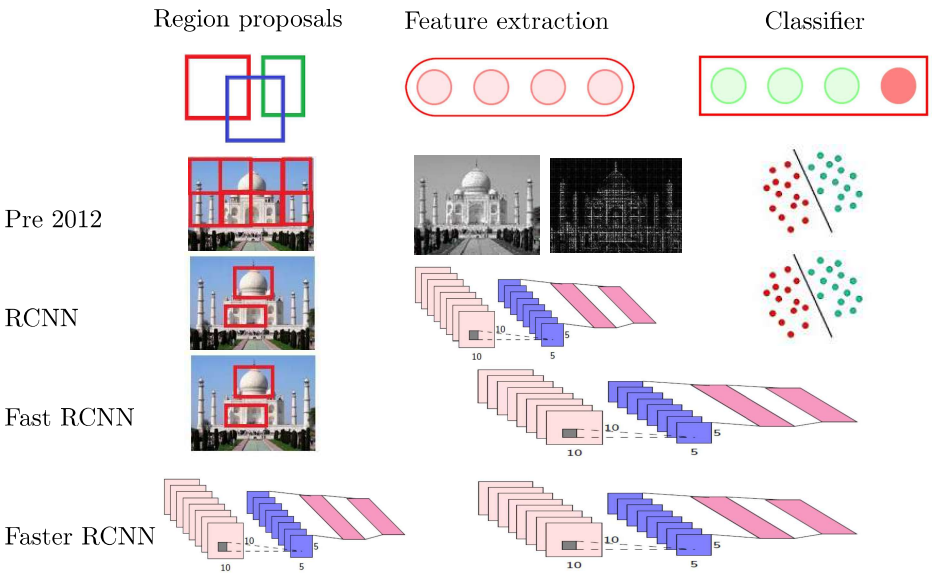
Task	Image classification	Object Detection
Output	Car	Car, exact bounding box containing car



- Let us see a typical pipeline for *object detection*
- It starts with a region proposal stage where we identify potential regions which may contain objects
- We could think of these regions as mini-images



- In addition we would also like to correct the proposed bounding boxes
- This is posed as a regression problem (for example, we would like to predict w^*, h^* from the proposed w and h)



Let us see how these three components have evolved over time

Propose all possible regions in the image of varying sizes (almost brute force)

Use handcrafted features (SIFT, HOG)

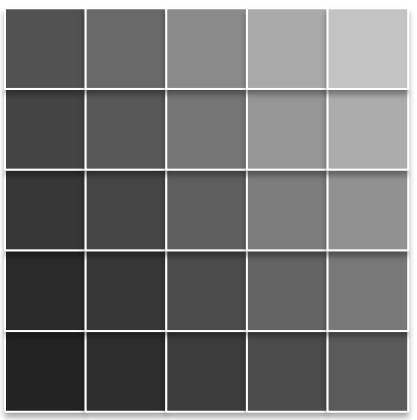
- Train a linear classifier using these features

We will now see three algorithms that progressively improve these components

HOG: What Happens to the Pixels?

Connect to SIFT: we again start from local gradients — but HOG does it densely.

START WITH PIXELS



Each square = ONE PIXEL

GRADIENT

magnitude = how strong
orientation = which direction

SIFT / ORB

Find interesting keypoints
Build a local gradient-based descriptor
→ sparse locations

HOG

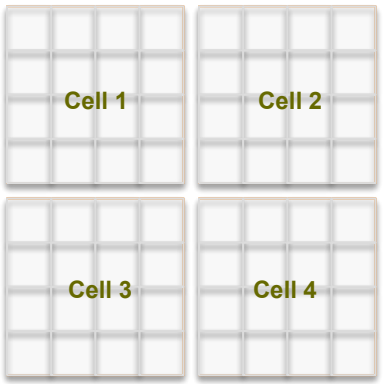
No keypoint detection
Look across the whole detection window
→ dense grid

SIFT: descriptor around keypoints | HOG: descriptor over a dense grid

From Pixels to a HOG Feature Vector

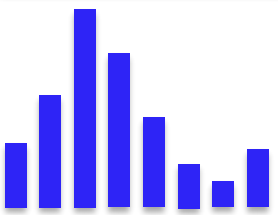
The hierarchy is the key: pixels → cells → blocks → one descriptor.

GROUP PIXELS INTO CELLS



A cell contains MANY pixels.

Each pixel casts a weighted vote into an orientation bin



gradient magnitude = vote strength

CELL DESCRIPTOR

= orientation histogram

“What edge directions are common?”

GROUP CELLS INTO BLOCKS

2 × 2 CELLS

Normalize neighboring cell histograms

4. COLLECT BLOCKS ACROSS THE WINDOW

[histogram | histogram | histogram | …] → one long HOG feature vector

How the HOG Detector Learns

HOG creates the features. A linear classifier learns which feature patterns mean “person”.

TRAINING

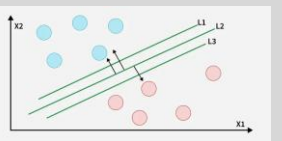
Positive person window

Negative background window

HOG

pixels → gradients → cells → histograms → blocks → vector

LINEAR SVM



to separate classes

TEST NEW WINDOW

x = HOG vector
score = $w^T x + b$
high score → object

TESTING: SLIDE THE WINDOW

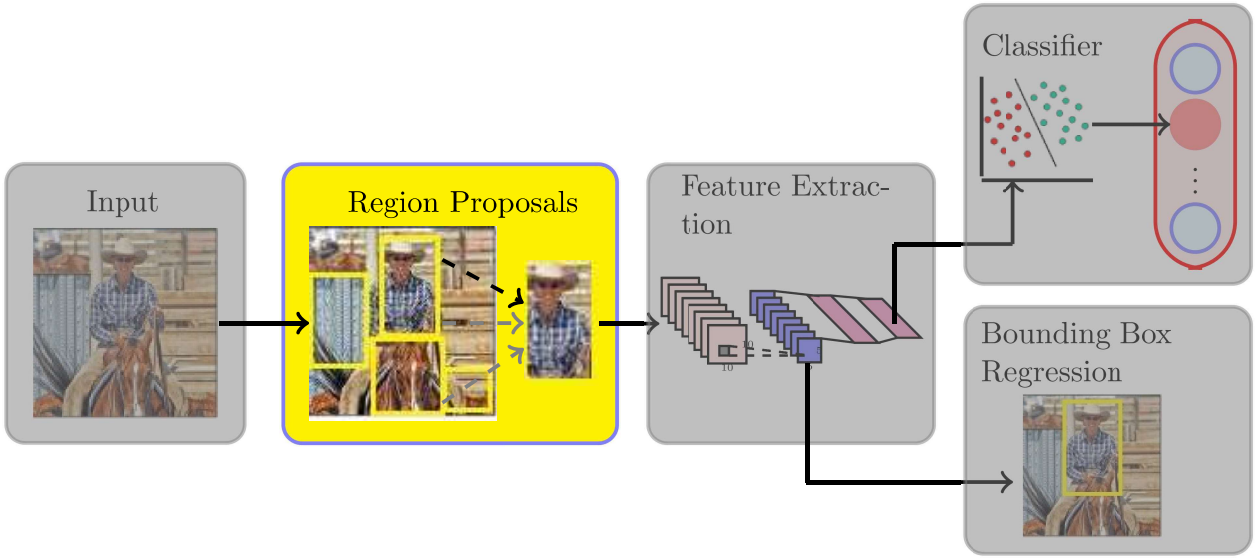


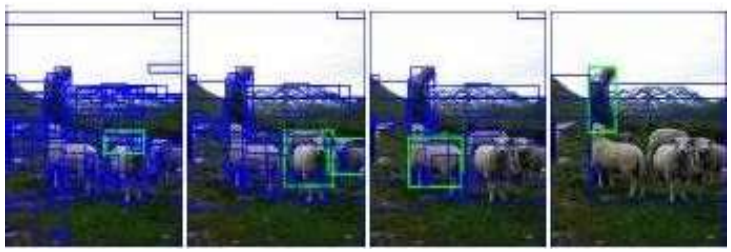
HOG + SVM
score this window

High scores → detections
NMS removes duplicates

SEARCH EVERYWHERE → HOG FEATURES → SVM → NMS

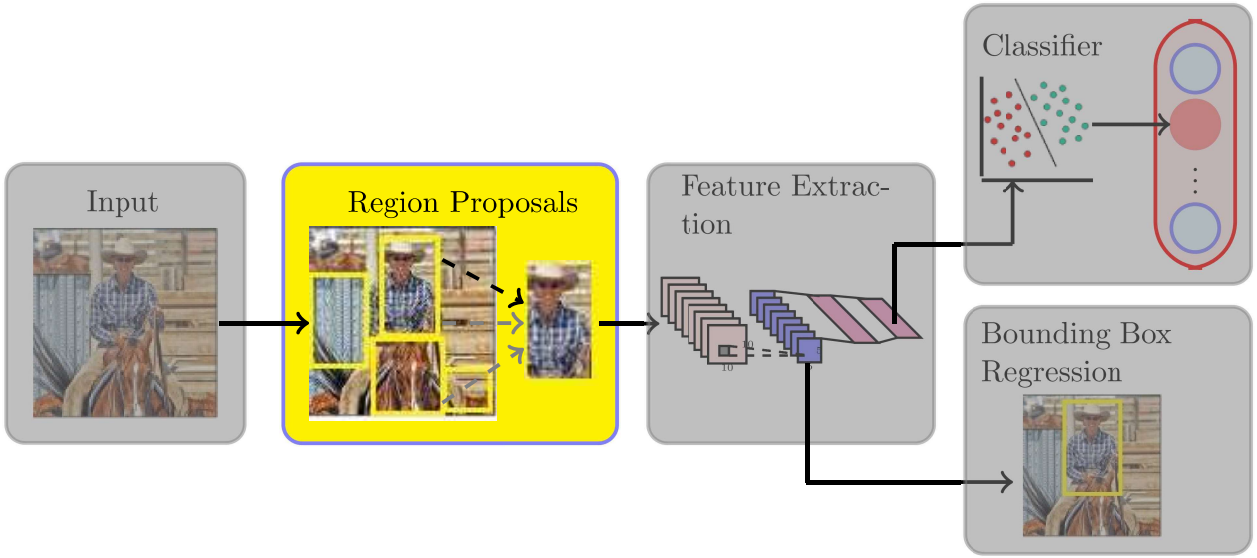
Module 6.2 : RCNN model for object detection

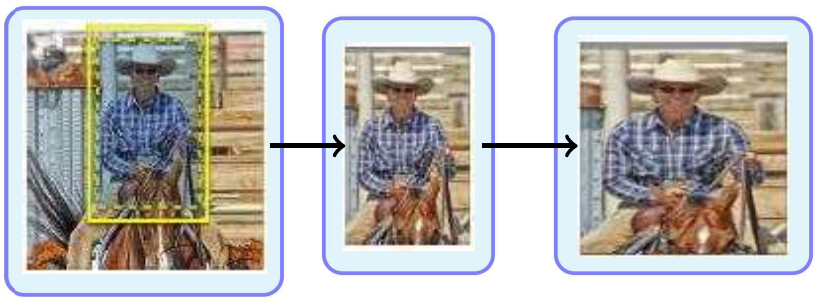




Selective Search for region proposals

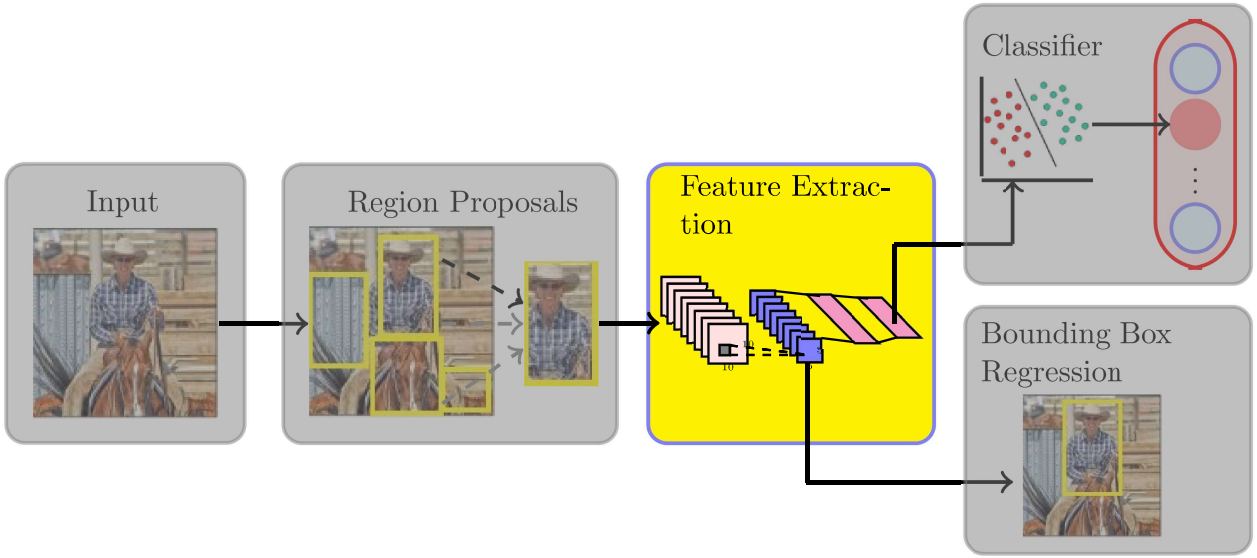
- Does hierarchical clustering at different scales
- For example the figures from left to right show clusters of increasing sizes
- Such a hierarchical clustering is important as we may find different objects at different scales

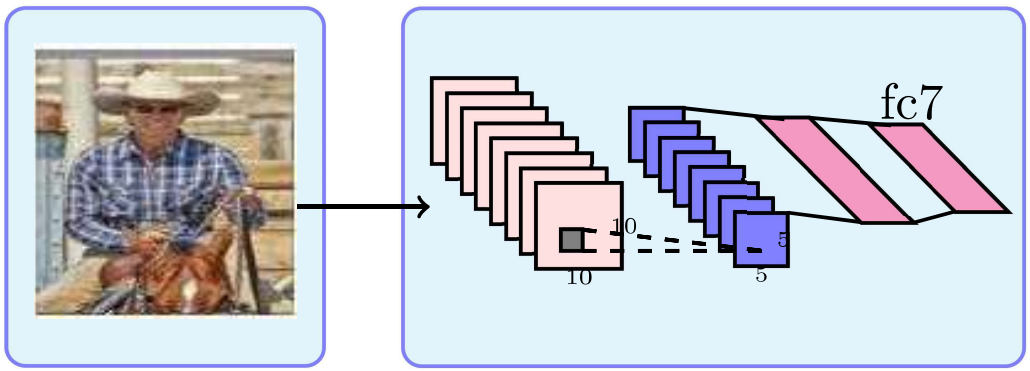




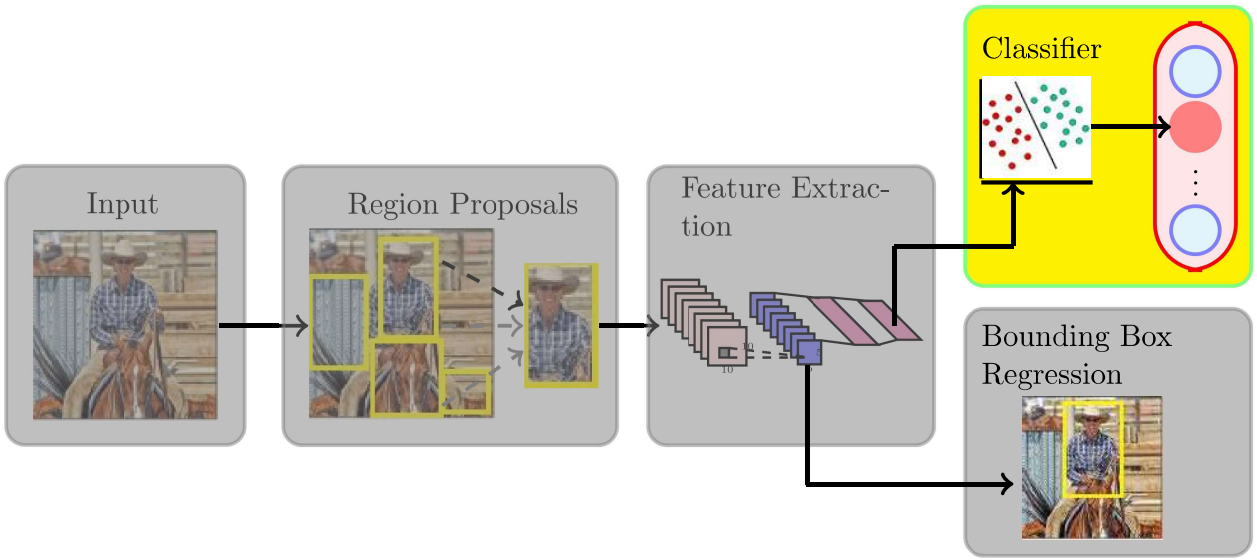
Proposed regions are cropped to form mini images

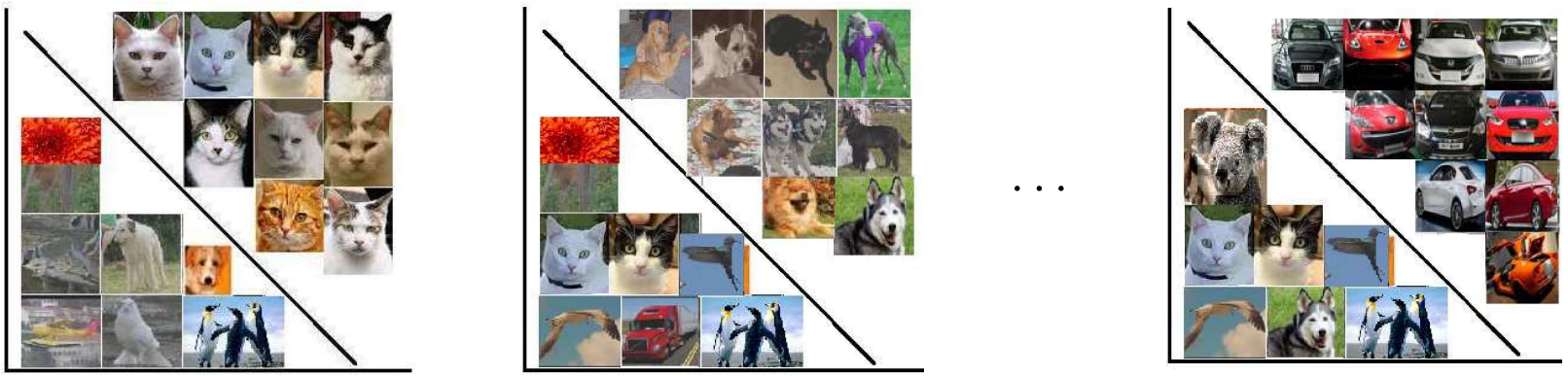
- Each mini image is scaled to match the CNN's (feature extractor) input size



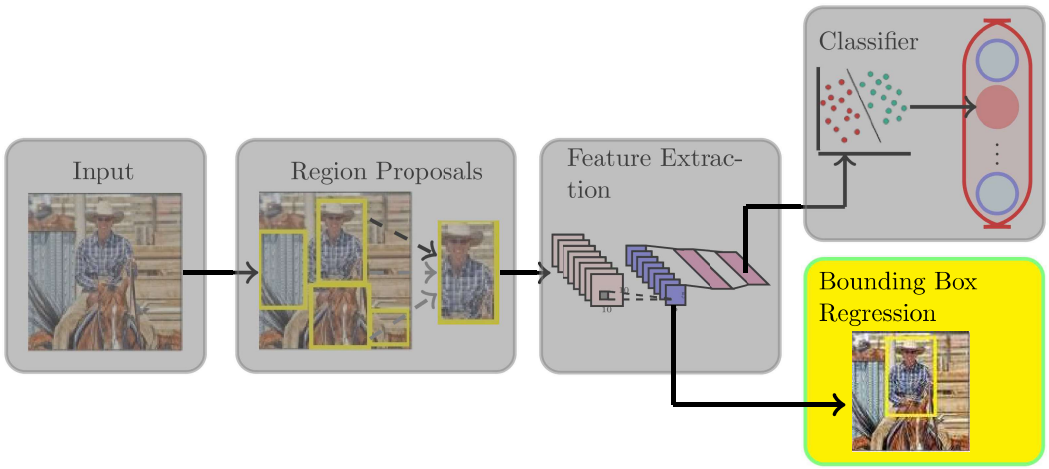


- For feature extraction any CNN trained for Image Classification can be used (AlexNet/ VGGNet etc.)
- Outputs from fc7 layer are taken as features
- CNN is fine tuned using ground truth (cropped) object images





- Linear models (SVMs) are used for classification (1 model per class)





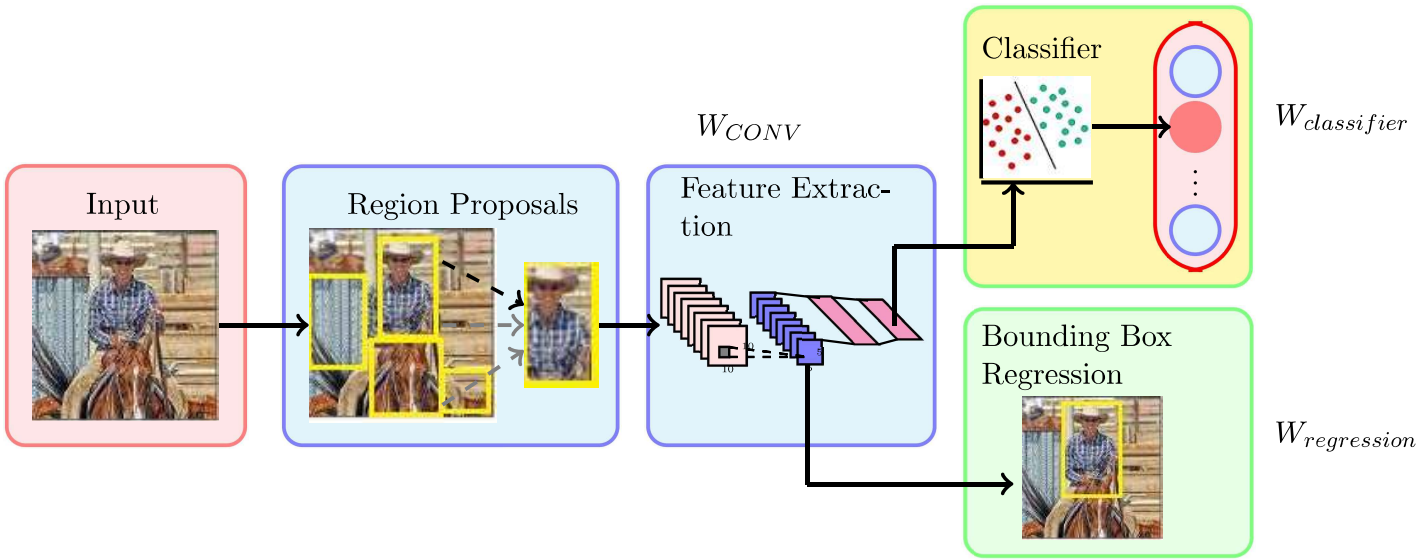
Proposed Box



True Box

z : features from pool5 layer of the network

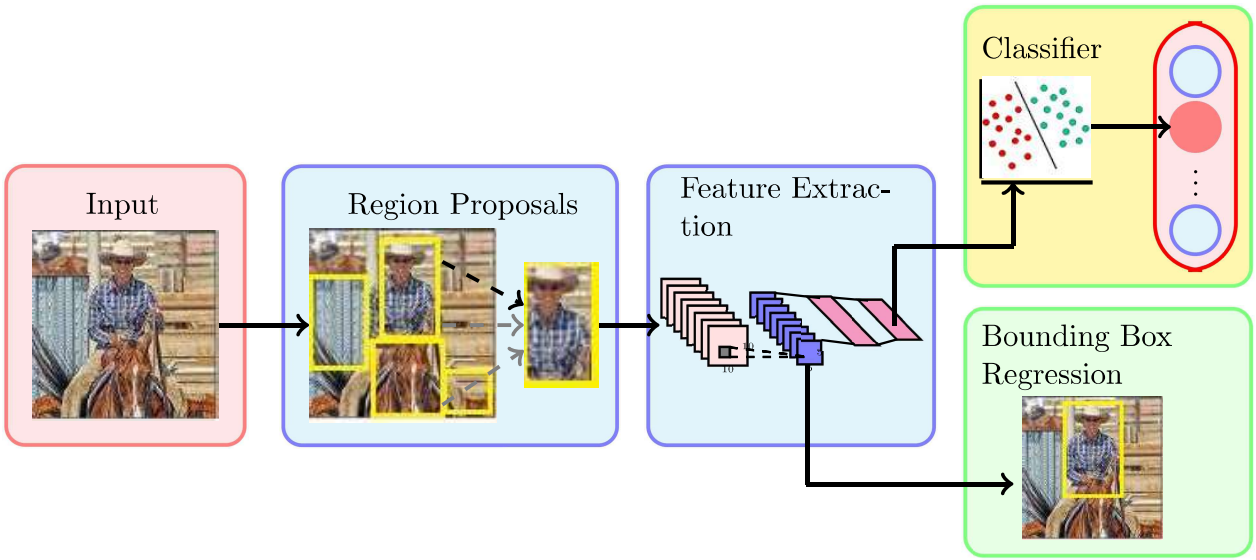
- The proposed regions may not be perfect
- We want to learn four regression models which will learn to predict x^*, y^*, w^*, h^*
- We will see their respective objective functions



What are the parameters of this model?

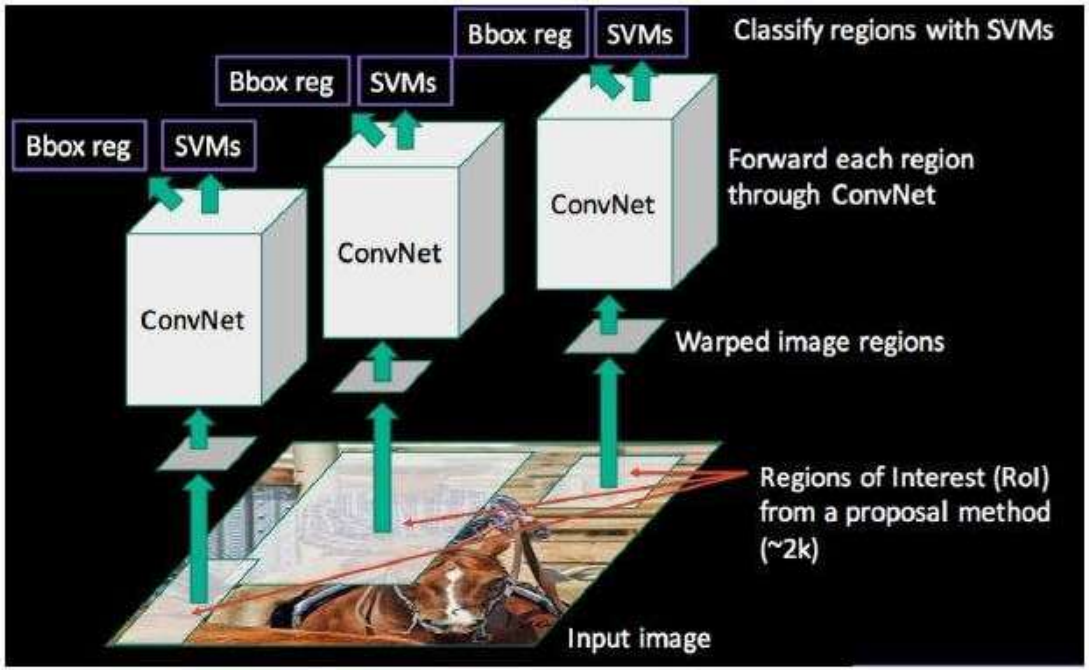
W_{CONV} is taken as it is from a CNN trained for Image classification (say on ImageNet)

- W_{CONV} is then fine tuned using ground truth (cropped) object images
- $W_{classifier}$ is learned using ground truth (cropped) object images
- $W_{regression}$ is learned using ground truth bounding boxes



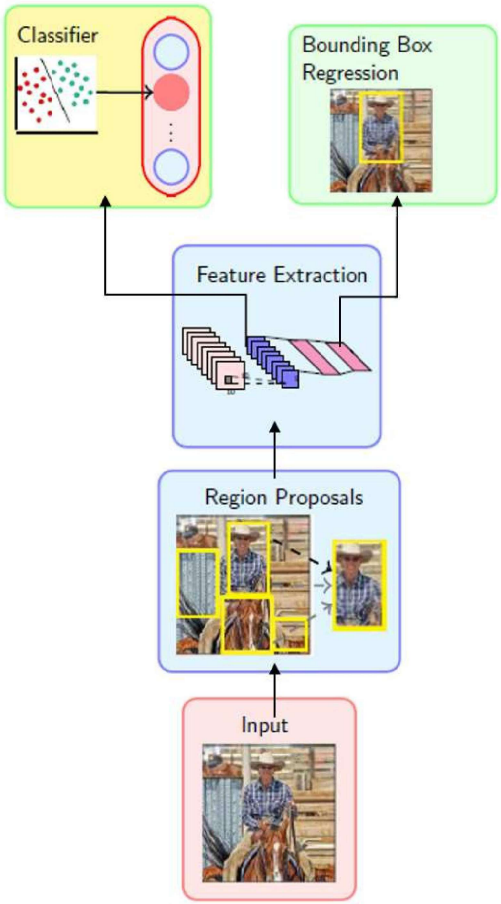
What is the computational cost for processing one image at test time?

Inference Time = Proposal Time + # Proposals × Convolution Time + # Proposals × classification + # Proposals × regression



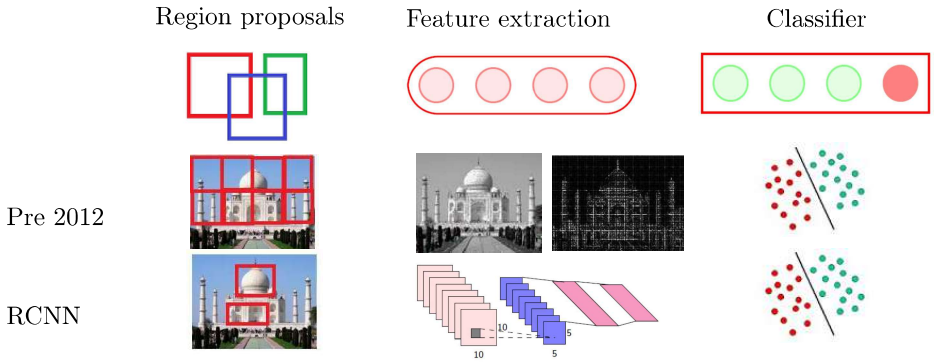
Source: *Ross Girshick*

- On average selective search gives 2K region proposal
- Each of these pass through the CNN for feature extraction
- Followed by classification and regression



- No joint learning
- Use ad hoc training objectives
  - Fine tune network with softmax classifier (log loss)
  - Train post-hoc linear SVMs (hinge loss)
  - Train post-hoc bounding-box regressors (squared loss)
- Training ($\approx$ 3 days) and testing (47s per image) is slow[1].
- Takes a lot of disk space

[1]Source: *Ross Girshick*
[1]Using VGG-Net

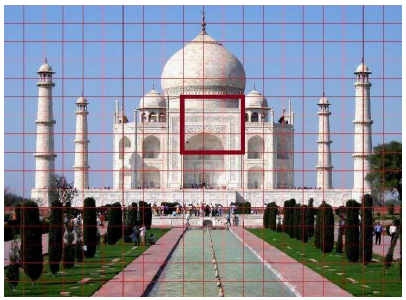


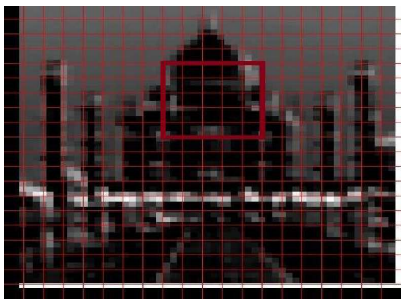
Region Proposals: Selective Search

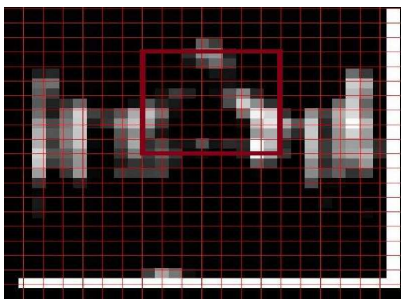
Feature Extraction: CNNs

Classifier: Linear

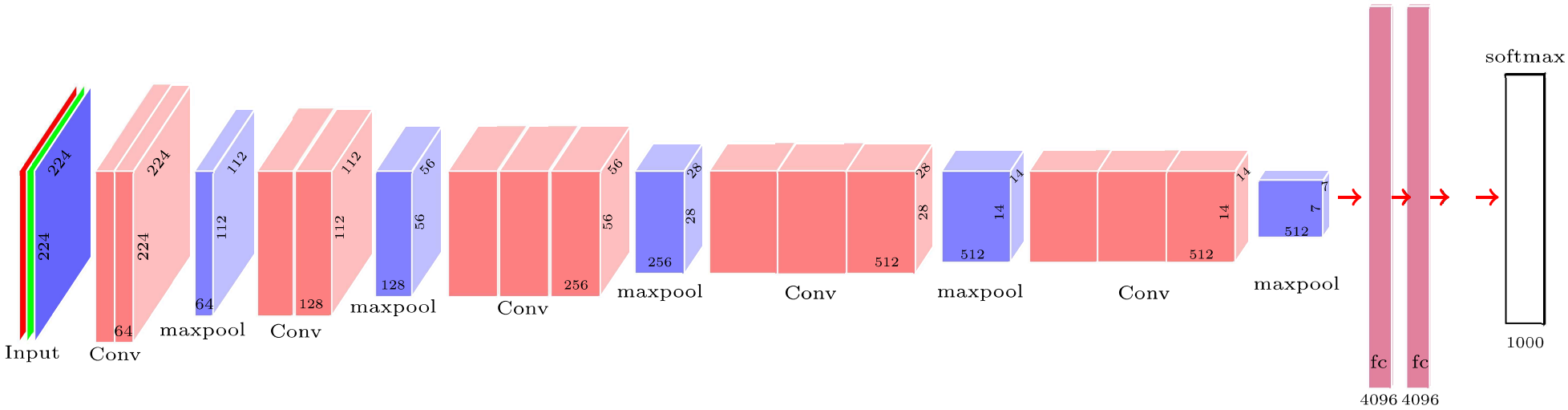
Module 6.3 : Fast RCNN model for object detection

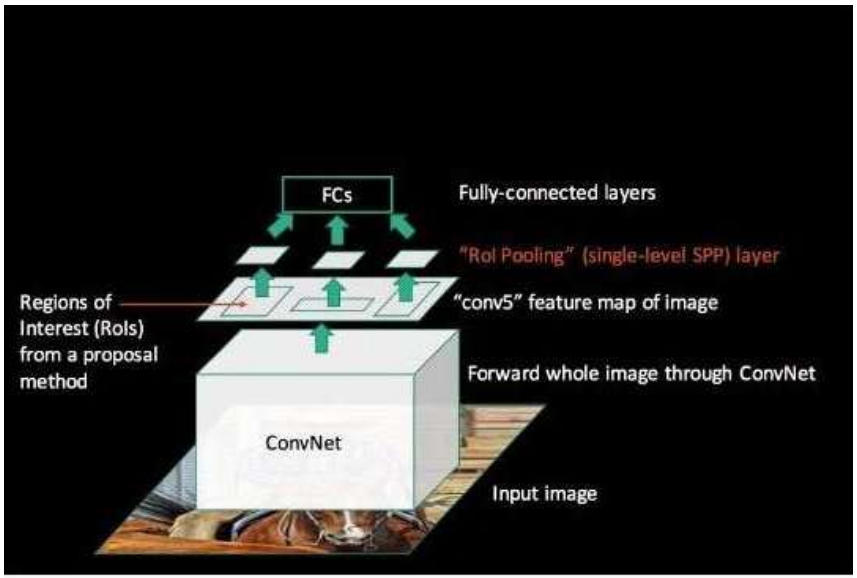






- Suppose we apply a 3×3 kernel on an image
- What is the region of influence of each pixel in the resulting output ?
- Each pixel contributes to a 5×5 region
- Suppose we again apply a 3×3 kernel on this output?
- What is the region of influence of the original pixel from the input ? (a 7×7 region)


Input
224
224
Conv
64
224
224
maxpool
64
112
112
Conv
128
112
112
maxpool
128
56
56
Conv
256
56
56
maxpool
256
28
28
Conv
512
28
28
maxpool
512
14
14
Conv
512
14
14
maxpool
512
7
7
fc
4096
fc
4096
softmax
1000

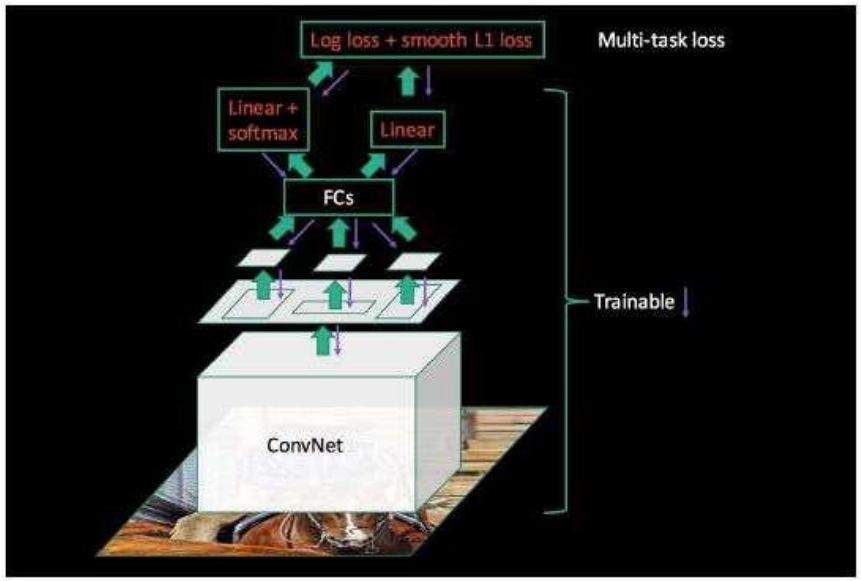


Source: *Ross Girshick*

Using this idea we could get a bounding box's region of influence on any layer in the CNN

The projected Region of Interest (RoI) may be of different sizes

- Divide them into k equally sized regions of dimension $H \times W$ and do max pooling in each of those regions to construct a k dimensional vector
- Connect the k dimensional vector to a fully connected layer
- This max pooling operation is call RoI pooling

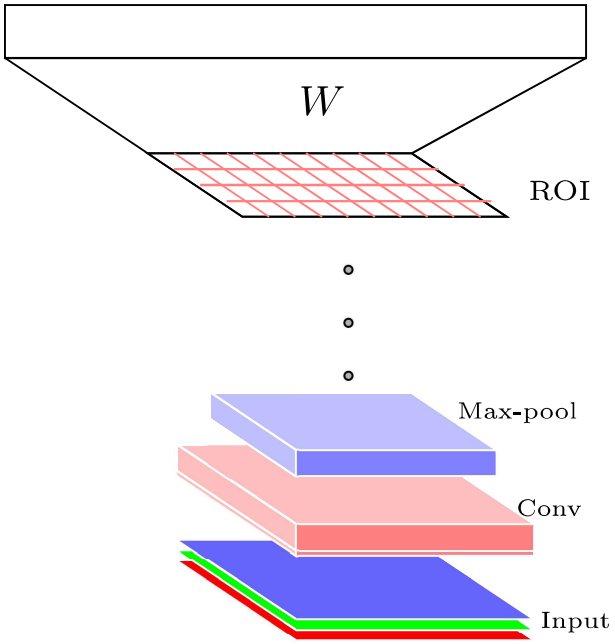


Source: *Ross Girshick*

Once we have the FC layer it gives us the representation of this region proposal

We can then add a softmax layer on top of it to compute a probability distribution over the possible object classes

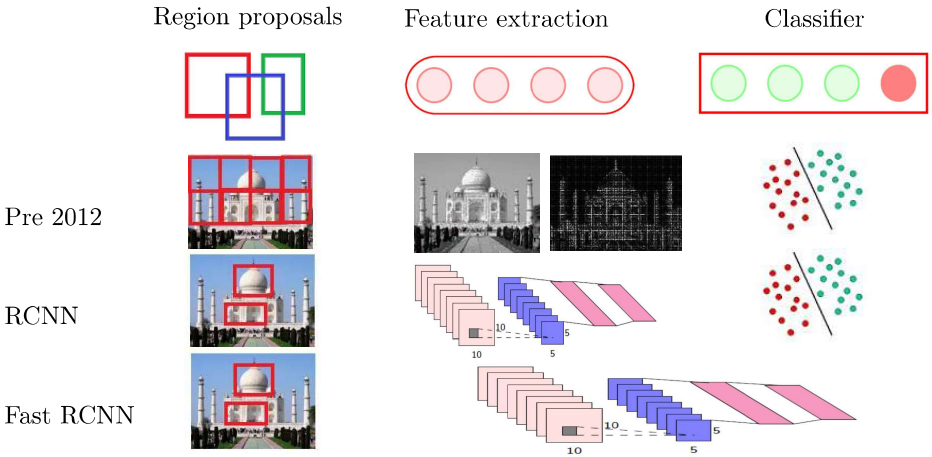
- Similarly we can add a regression layer on top of it to predict the new bounding box (w^*, h^*, x^*, y^*)



- Recall that the last pooling layer of VGGNet-16 results in an output of size $512 \times 7 \times 7$
- We replace the last max pooling layer by a RoI pooling layer

  We set $H = W = 7$ and divide each of these RoIs into $(k = 49)$ regions
- We do this for every feature map resulting in an ouput of size 512×49
- This output is of the same size as the output of the original max pooling layer

- It is thus compatible with the dimensions of the weight matrix connecting the original pooling layer to the first

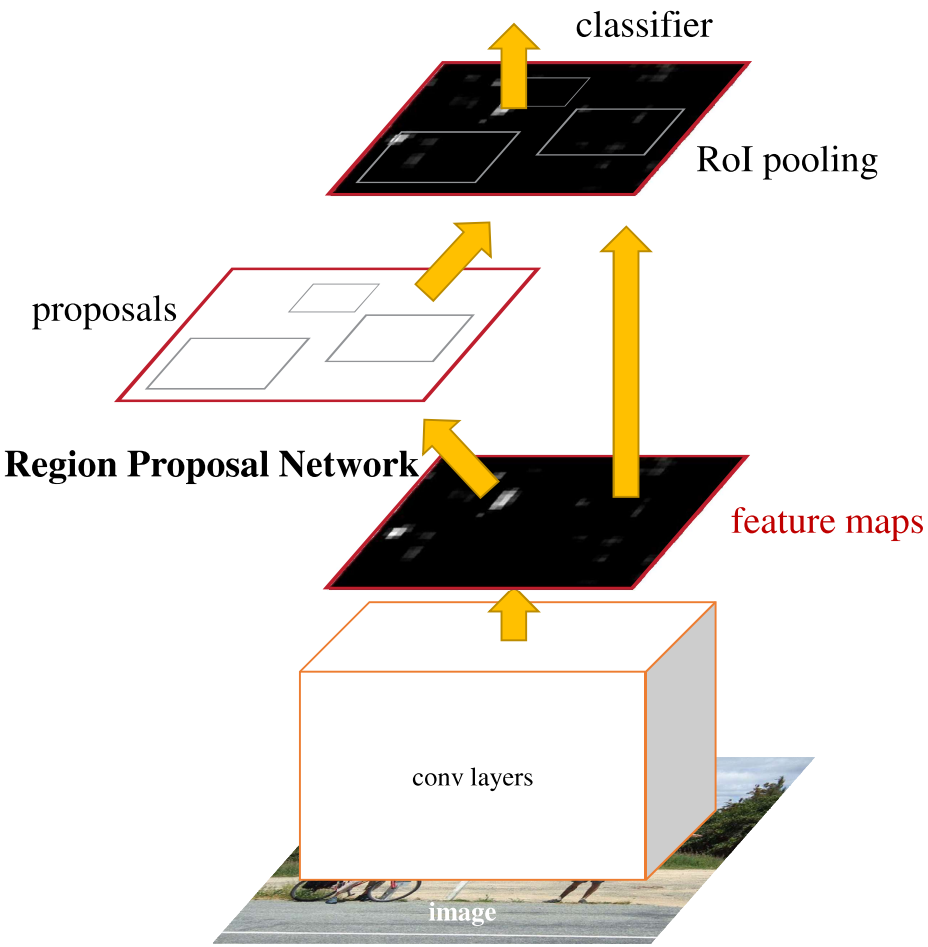


Region Proposals: Selective Search

Feature Extraction: CNN

Classifier: CNN

Module 6.4 : Faster RCNN model for object detection

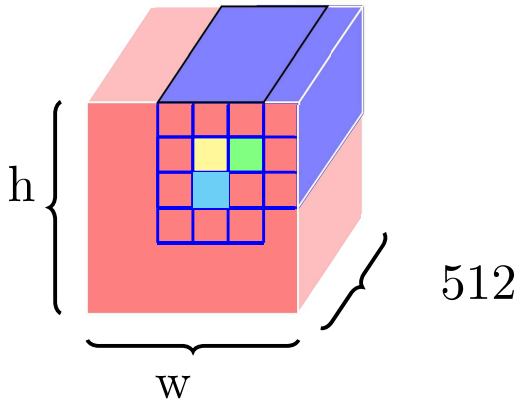


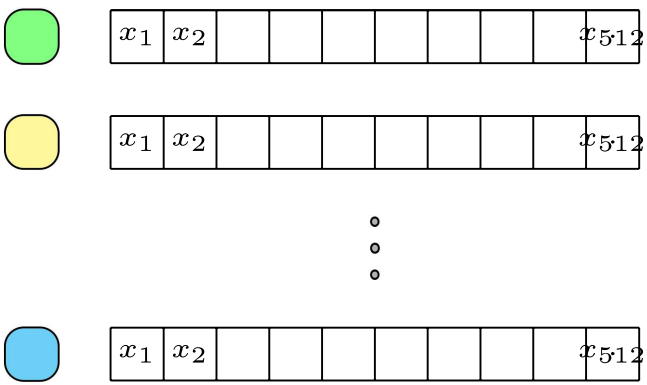
So far the region proposals were being made using Selective Search algorithm

Idea: Can we use a CNN for making region proposals also?

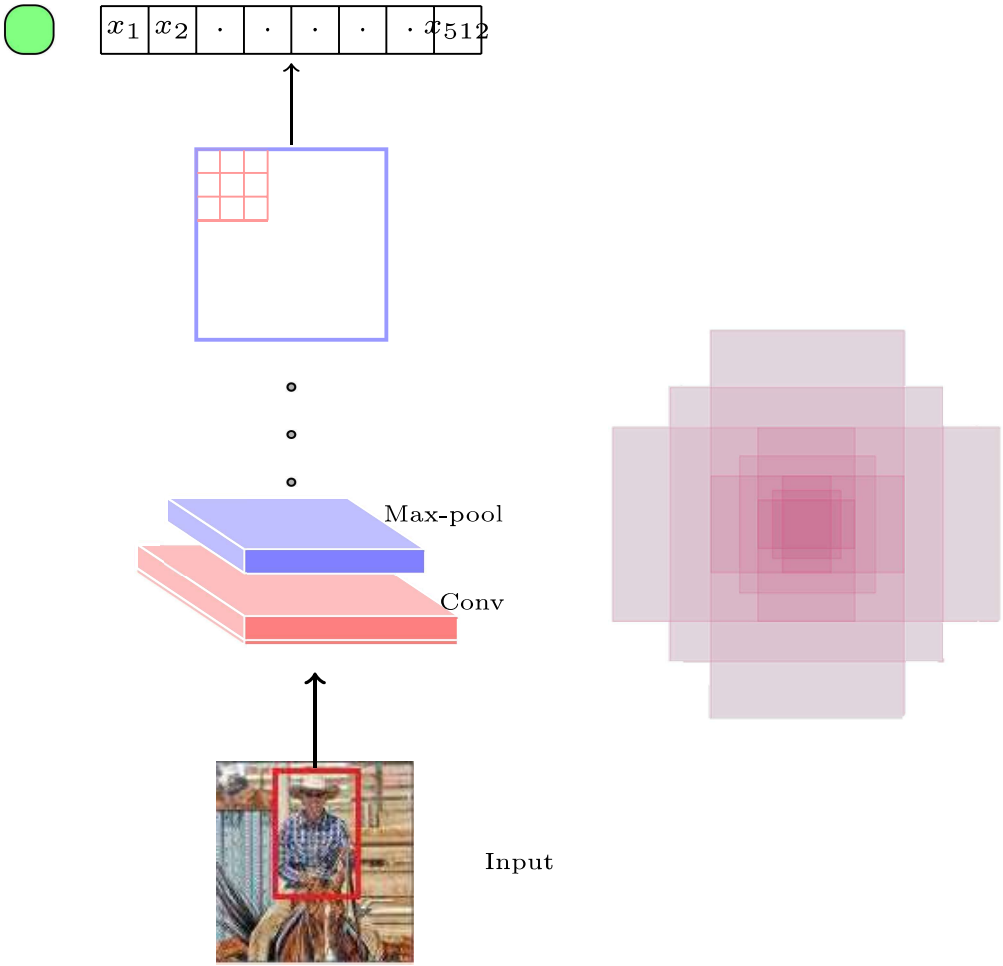
How? Well it's slightly tricky

We will illustrate this using **VGGNet**

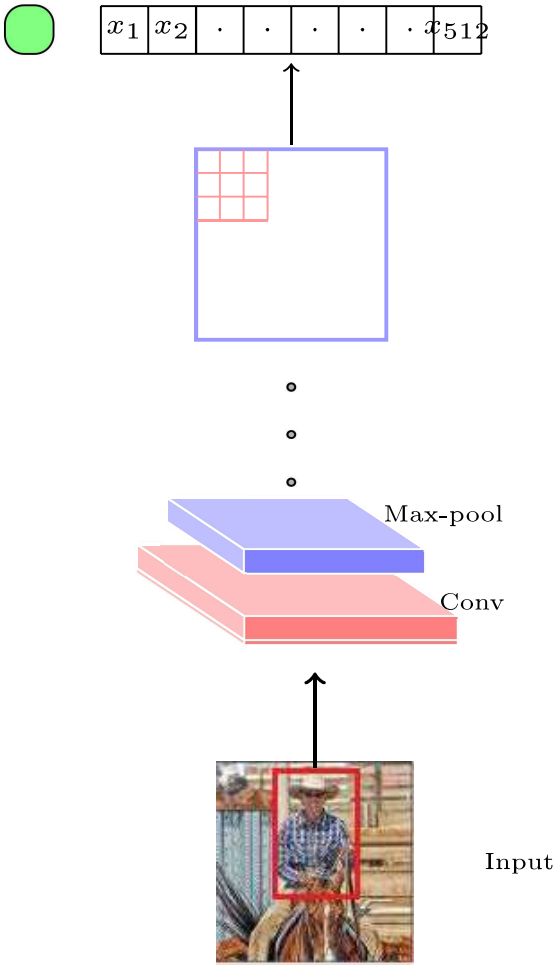


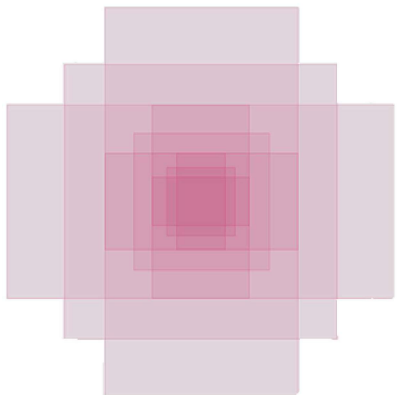


- Consider the output of the last convolutional layer of VGGNet
- Now consider one cell in one of the 512 feature maps
- If we apply a 3×3 kernel around this cell then we will get a 1D representation for this cell
- If we repeat this for all the 512 feature maps then we will get a 512 dimensional representation for this position
- We use this process to get a 512 dimensional representation for each of the $w \times h$ positions

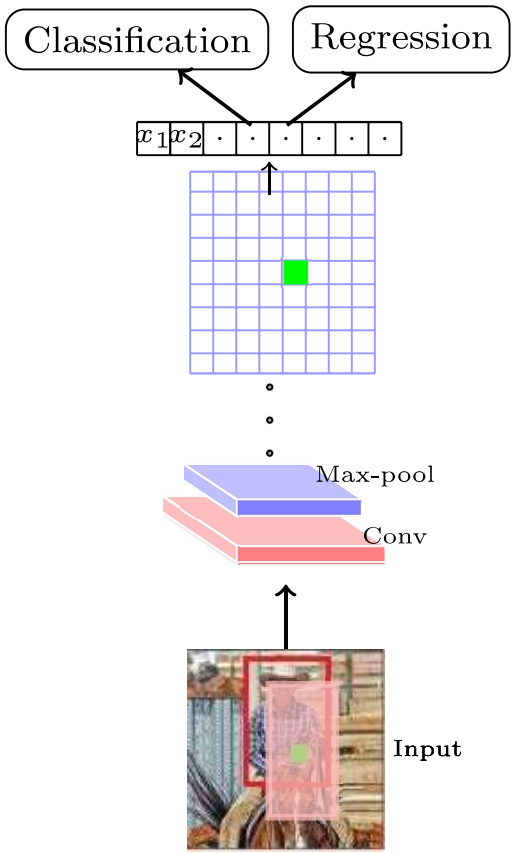


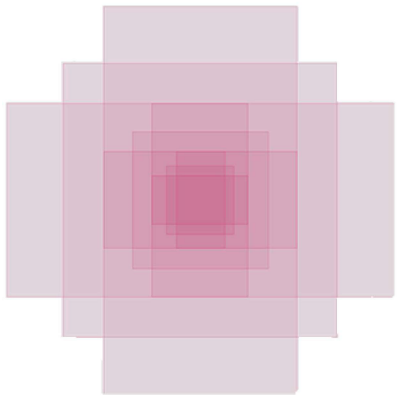
- We now consider k bounding boxes (called anchor boxes) of different sizes & aspect ratio
- We are interested in the following two questions:
- Given the $512d$ representation of a position, what is the probability that a given anchor box centered at this position contains an object? (Classification)
- How do you predict the true bounding box from this anchor box? (Regression)



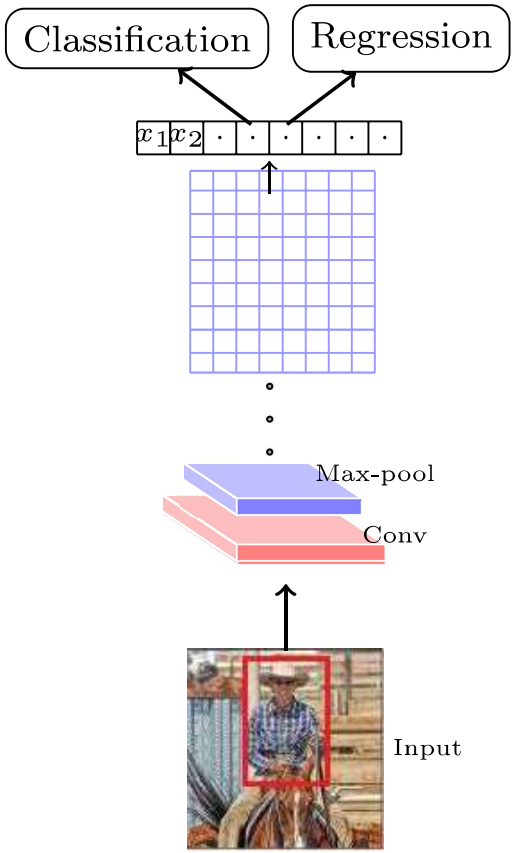


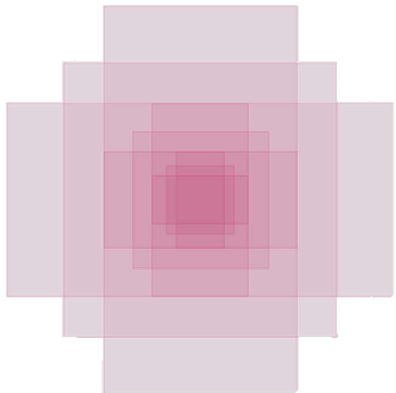
- We train a classification model and a regression model to address these two questions
- How do we get the ground truth data?
- What is the objective function used for training?





- Consider a ground truth object and its corresponding bounding box
- Consider the projection of this image onto the conv5 layer
- Consider one such cell in the output
- This cell corresponds to a patch in the original image
- Consider the center of this patch
- We consider anchor boxes of different sizes
- For each of these anchor boxes, we would want the classifier to predict 1 if this anchor box has a reasonable overlap (IoU > 0.7) with the true grounding box



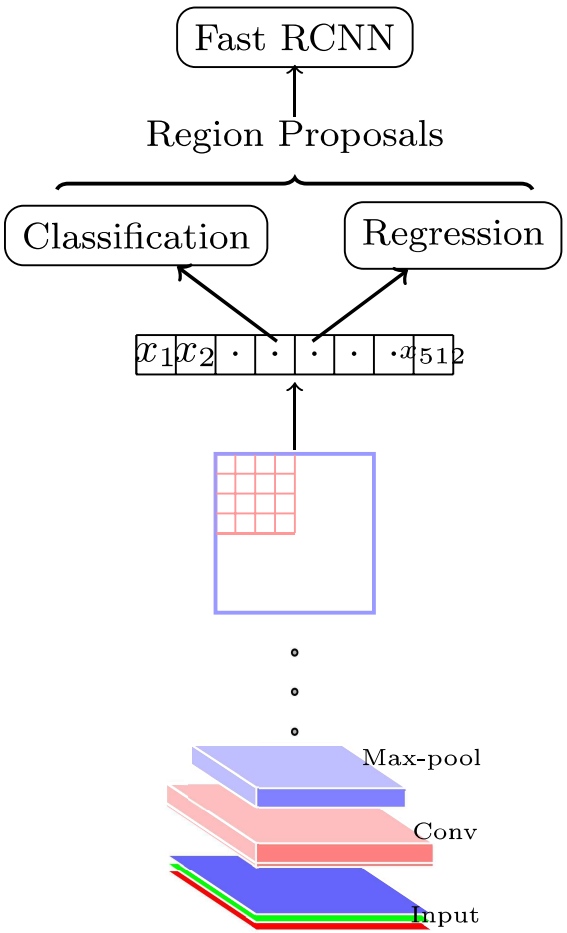


- We train a classification model and a regression model to address these two questions
- How do we get the ground truth data?
- What is the objective function used for training?

- The full network is trained using the following objective.

$$\mathscr{L}(p_i, t_i) = \frac{1}{N_{cls}} \sum_i \mathscr{L}_{cls}(p_i, p_i^*) + \frac{\lambda}{N_{reg}} \sum_i p_i^* \mathscr{L}_{reg}(t_i, t_i^*)$$

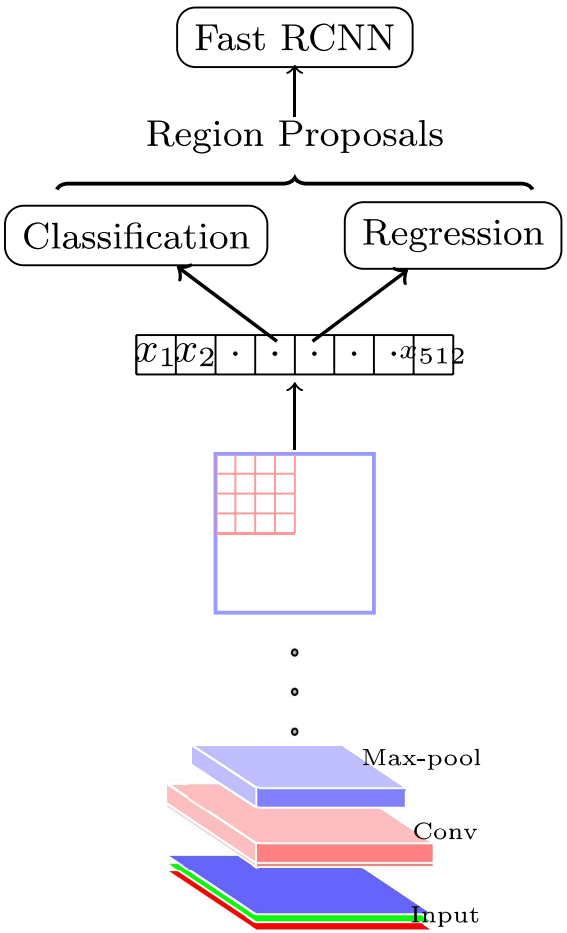
$$\begin{aligned}
p_i^* &= 1 \quad \text{if anchor box contains ground truth object} \\
&= 0 \quad \text{otherwise} \\
p_i &= \text{predicted probability of anchor box containing an object} \\
N_{cls} &= \text{batch-size} \\
N_{reg} &= \text{batch-size} \times k \\
k &= \text{anchor boxes}
\end{aligned}$$



So far we have seen a CNN based approach for region proposals instead of using selective search

We can now take these region proposals and then add fast RCNN on top of it to predict the class of the object

And regress the proposed bounding box

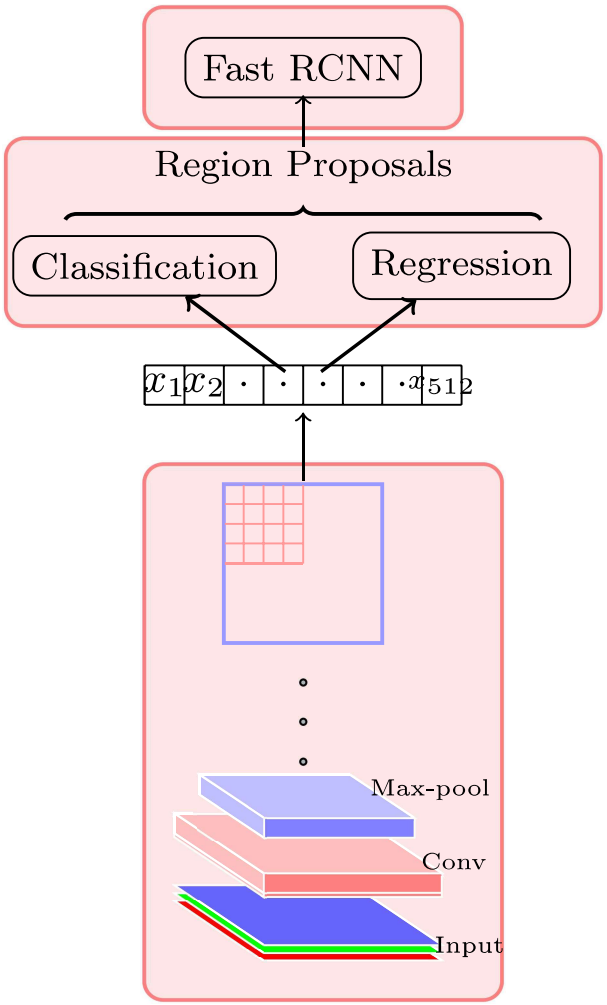


But the fast RCNN would again use a VGG Net

Can't we use a single VGG Net and share the parameters of RPN and RCNN

Yes, we can

In practice, we use a 4 step alternating training process



Faster RCNN:Training

Fine-tune RPN using a pre-trained ImageNet network

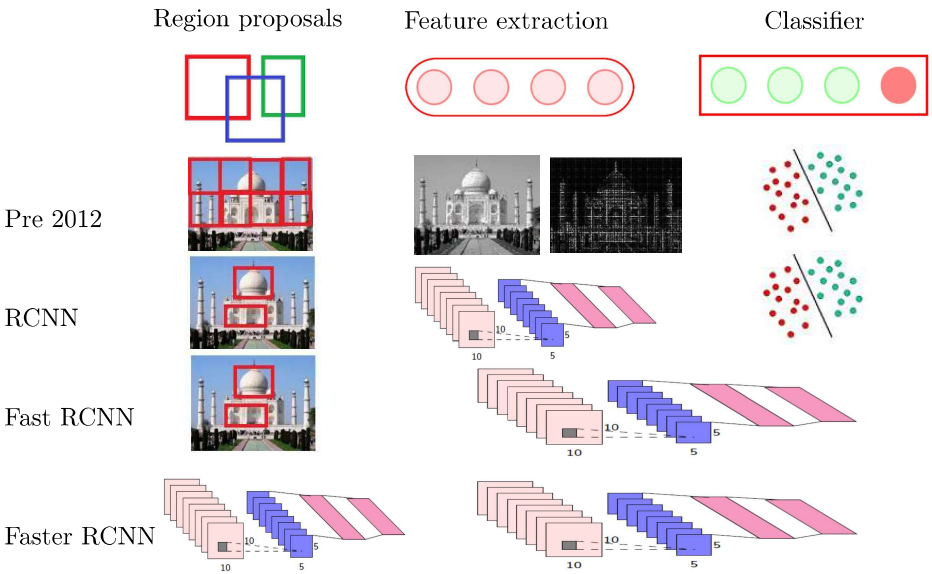
Fine-tune fast RCNN from a pre-trained ImageNet network using bounding boxes from step 1

Keeping common convolutional layer parameters fixed from step 2, fine-tune RPN (post conv5 layers)

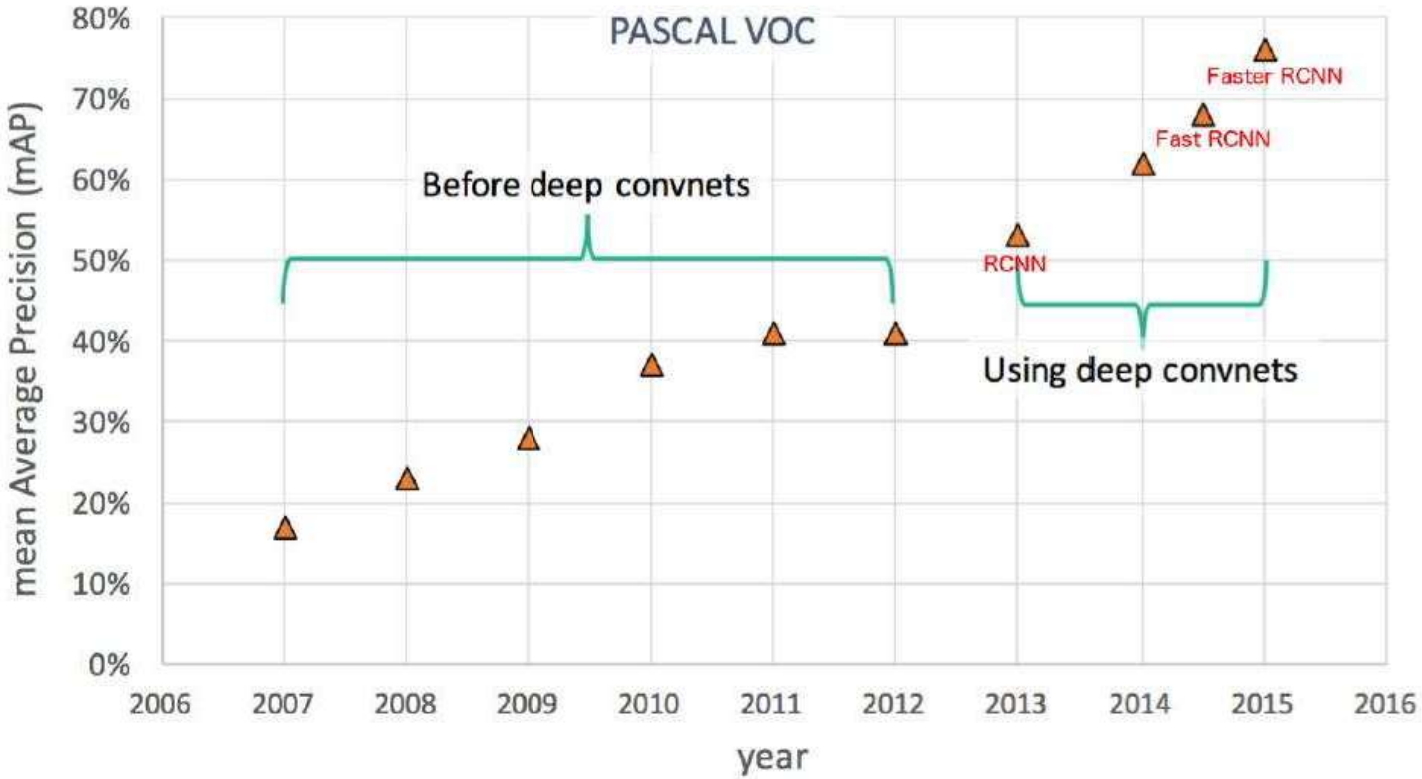
Keeping common convolution layer parameters fixed from step 3, fine-tune fc layers of fast RCNN

Faster RCNN and RPN are the basis of several 1st place entries in the ILSVRC and COCO tracks on :

- Imagenet detection
- COCO Segmentation
- Imagenet localization
- COCO detection



- **Region Proposals:** CNN
- **Feature Extraction:** CNN
- **Classifier:** CNN



Object Detection Performance

Source: Ross Girshick